

Discovery Executive Summary

Project: discovery-24-jul-003 · Generated: 24/07/2026, 16:04:01

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 1 discovery analysis run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating	Hotspot Score
1	Architecture & Design Analysis	HIGH RISK	—

1. Architecture & Design Analysis

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk Missing Service Layer (H2), Missing Repository Pattern (H3), Direct SQL/ORM in Controllers (H6), Domain Boundary Violations (H8), Shared Database Coupling (H9), and Dual-Runtime Drift (H10).

EXECUTIVE SUMMARY

Klearcom is a modular monolith for Discovery (IVR) and Connect (TFN) with three runtime layers: Laravel 12 API (`backend/`), React 19 SPA (`frontend/`), and a parallel Node Express `dev-api` used for local development. Layers covered: **backend** (20 PHP application files under `backend/app/` plus routes), **frontend** (14 TS/JSX source files under `frontend/src/`), and **dev-api** (6 JS modules). The dominant risk is **missing application/repository boundaries**: controllers and Express handlers own Eloquent/in-memory store access (25 Eloquent call sites in controllers alone), while declared `app/Modules/{Discovery,Connect}` folders contain only `AGENTS.md` stubs. Cross-domain coupling through `DashboardController`, `LegacyReportController`, `RealTimeTestService`, and shared Mongo collections amplifies change cost. Frontend is comparatively healthier (shared `api/client.ts`, React Query) but still carries legacy class/unbounded widgets without error boundaries.

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	62 LOC	GOOD
H2	Missing Service Layer	Controllers accessing repos/models	<10	10–20	>20	25 Eloquent sites in 4 controllers	HIGH RISK
H3	Missing Repository Pattern	Direct DB access points	<10	10–20	>20	32+ (0 repositories)	HIGH RISK
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	GOOD
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	2 (LegacyDataMapper , store.buildTree)	MODERATE
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	~22% (7/32 Eloquent sites in services)	HIGH RISK
H7	God Classes	Classes >1000 LOC	0	1–3	>3	0 (largest: server.js 224 LOC)	GOOD
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	8+	HIGH RISK
H9	Shared Database Coupling	Tables shared across domains	<10%	10–30%	>30%	~38% (3/8 stores shared via Mongo)	HIGH RISK
H10	Dual-Runtime Drift (additional)	Parallel domain impls Laravel vs Node	0	1–2	>2	4+ duplicated flows	HIGH RISK
F1	Business Logic in Components	Avg LOC per component	<150	150–300	>300	74 LOC	GOOD
F2	Missing Frontend Service/Data Layer	Components w/ inline API calls	<10	10–20	>20	6 components + 1 hook	GOOD
F3	God / Oversized Components	Components >400 LOC	0	1–3	>3	0 (max ConnectPage 208 LOC)	GOOD
F4	Prop Drilling / Global State Abuse	Max prop-drilling depth	≤2	3–4	>4	≤2 (uiStore focused)	GOOD

F5	Legacy / Inconsistent Component Patterns	Legacy-pattern components	0	1–10	>10	2	MODERATE
----	--	---------------------------	---	------	-----	---	----------

1.4 Actions Required

Hotspot	Action	Rating	Priority
H2 Missing Service Layer	Extract Discovery/Connect/Dashboard application services; centralize reachability + tree builders; thin controllers to HTTP adapters	HIGH RISK	CRITICAL
H3 Missing Repository Pattern	Introduce repository interfaces for Eloquent models and Mongo collections; bind in <code>AppServiceProvider</code> ; zero direct ORM in controllers	HIGH RISK	CRITICAL
H5 Shared Utility Abuse	Replace <code>LegacyDataMapper</code> extract mapping and duplicate <code>buildTree</code> with domain services (<code>IvrTreeBuilder</code> , explicit DTOs)	MODERATE	MEDIUM
H6 Direct SQL/ORM in Controllers	Move all Eloquent/Mongo queries out of controllers (incl. Node diagnostics route) behind repositories	HIGH RISK	HIGH
H8 Domain Boundary Violations	Populate <code>Modules/{Discovery,Connect}</code> with real code; ban cross-model imports; dashboard via published interfaces only	HIGH RISK	CRITICAL
H9 Shared Database Coupling	Split shared Mongo collections per domain or wrap with per-module ACLs; keep MariaDB table ownership	HIGH RISK	HIGH
H10 Dual-Runtime Drift	Consolidate Laravel vs Node <code>dev-api</code> duplicated flows into one source of truth	HIGH RISK	CRITICAL
F5 Legacy Component Patterns	Remove/rewrite <code>LegacyMonitorPoller</code> + <code>LegacyDashboardWidget</code> ; add root Error Boundary; colocate under <code>frontend/src/modules/</code>	MODERATE	MEDIUM

1.5 Expected Outcomes

- Controllers become thin HTTP adapters; Discovery/Connect workflows are unit-testable via application and domain services without booting Laravel HTTP kernel.
- Repository interfaces enable swapping MariaDB/Mongo implementations and aligning `dev-api` with the same contracts.
- Bounded contexts with published interfaces stop silent cross-domain regressions when IVR or TFN schemas change.
- Split or ACL-wrapped Mongo collections let each module evolve retention, indexes, and payload shapes independently.
- A single API runtime eliminates Laravel/Node drift so frontend testing matches production Docker behavior.